

Final Incident Report

Google Cloud Security Incident Response — Simulated Breach Investigation and Remediation

Organization Profile (Training Scenario): Cymbal Retail

Environment: qwiklabs-gcp-04-31c75ab463d3 | Lab Date: June 22, 2026

Prepared By	Seelam Narrammagari Amogh
Program	B.Tech, Cyber Security
Institution	Vignan's Institute of Information Technology, Visakhapatnam
Role	Junior Cloud Security Analyst (Capstone Exercise)
LinkedIn	linkedin.com/in/amoghsn8
Report Classification	Training / Educational — Simulated Lab Environment
Document Version	1.0 — Final

Prepared in accordance with the Google Cloud Skills Boost Security Capstone and presented using professional incident-response reporting conventions.

Table of Contents

Table of Contents.....	2
Executive Summary.....	3
Incident Overview.....	4
Training Scenario Context.....	4
Scope of the Exercise.....	4
Objectives.....	4
Investigation Findings.....	5
Security Command Center Overview.....	5
PCI DSS Compliance Report.....	5
Resource-Level Findings.....	6
Consolidated Findings Table.....	7
Response and Remediation.....	8
Containment and Eradication Measures.....	8
Recovery Measures.....	8
Detailed Remediation Activities.....	10
Task 1: Security Analysis and Investigation.....	10
Task 2: Compute Engine Recovery.....	11
Task 3: Cloud Storage Security Hardening.....	13
Task 4: SSH Access Restriction Using Tag-Scoped Firewall Rules.....	14
Task 5: Firewall Hardening and Logging.....	15
Task 6: Compliance Verification.....	18
Recommendations.....	19
Security Improvements Achieved.....	20
Lessons Learned.....	21
Conclusion.....	22

Executive Summary

This report documents a simulated Google Cloud security incident response exercise conducted as part of the Google Cloud Skills Boost Security Capstone Project, performed against a training environment representing a fictional retail organization, Cymbal Retail. The purpose of the exercise was to identify, investigate, and remediate a representative set of cloud misconfigurations using Google Cloud's native security tooling, primarily Security Command Center (SCC). No real customer data or production infrastructure was involved; all findings, resources, and remediation actions described below occurred within an isolated Qwiklabs training project (qwiklabs-gcp-04-31c75ab463d3).

At the outset of the exercise, the SCC Risk Overview surfaced multiple misconfigurations distributed across Firewall, Compute Engine, Cloud Storage, and supporting resource types, as shown in Figure 1. The Firewall resource type alone accounted for two High-severity and four Medium-severity findings, while Compute Engine accounted for one High-severity and three Medium-severity findings, and Cloud Storage carried one High, one Medium, and one Low-severity finding. In parallel, a PCI DSS 3.2.1 compliance scan (Figure 2) flagged several controls as Non-Compliant, most notably control families 10.1 and 10.2 (each tied to 29 findings related to audit logging and trail requirements), along with 1.2.1, 7.1.2, and 7.2, which govern network access restriction and least-privilege data access. SCC Findings (Figure 3) specifically identified an open RDP port (High severity) and disabled firewall rule logging (Medium severity) as active, unresolved issues.

Collectively, these findings indicated that the environment's attack surface was wider than necessary: a virtual machine instance reachable without sufficiently scoped network rules, broad default firewall rules permitting RDP and SSH traffic from any source, disabled logging on key firewall rules, and storage and IAM configurations that had not yet been hardened to the principle of least privilege. While the lab did not stage an actual exploitation event, these conditions are the same class of weaknesses commonly leveraged in real-world breaches — particularly overly permissive remote-access ports and unmonitored network boundaries — which is why the capstone exercise treats them as a simulated incident requiring full investigation and remediation.

The response effort proceeded through investigation, containment, recovery, and verification phases. The Compute Engine resource (cc-app-01) was stopped and ultimately decommissioned, and a replacement, hardened instance (cc-app-02) was provisioned to take its place, consistent with a clean-rebuild containment strategy. On Cloud Storage, the affected bucket's permissions were reviewed and tightened: Public Access Prevention was confirmed as actively enforced, and the bucket was migrated from fine-grained, object-level ACLs to uniform bucket-level access, with the allUsers principal explicitly removed from the bucket's IAM bindings. On the network layer, the broad default-allow-rdp and default-allow-ssh firewall rules were deleted and replaced with a narrowly scoped limit-ports rule restricting TCP/22 access to instances carrying the cc network tag. Logging was then explicitly enabled on both the new limit-ports rule and the pre-existing default-allow-internal rule to restore audit visibility consistent with PCI DSS logging requirements.

By the conclusion of the exercise, the environment's exposure had been measurably reduced: unscoped administrative ports were closed, public storage access paths were eliminated, firewall logging was restored, and the compromised-pattern VM was replaced with a hardened instance. This report documents each finding, the corresponding remediation action, the security rationale behind it, and the residual recommendations needed to sustain this improved posture going forward.

Incident Overview

Training Scenario Context

This exercise is modeled on the Google Cloud Skills Boost capstone scenario for Cymbal Retail, a fictional retail organization used throughout Google Cloud's security training curriculum. The scenario frames the learner as a newly onboarded Junior Cloud Security Analyst tasked with using Security Command Center and related GCP security tooling to triage a set of pre-seeded misconfigurations within a sandboxed Google Cloud project, then remediate them using native console workflows.

It is important to state plainly what this report is and is not. It is a record of a structured, graded training lab: the misconfigurations were deliberately provisioned into the lab environment for the learner to discover and fix, and no unauthorized third party accessed any data during this exercise. It is not a record of an actual confirmed data breach against a live production system. The report nonetheless follows professional incident-response documentation conventions — including severity classification, root-cause analysis, containment and recovery phases, and verification — because that structure is the explicit objective of the capstone assignment and reflects how a real investigation of comparable findings would be documented.

Scope of the Exercise

The scope of this exercise covered four Google Cloud resource domains within the lab project:

- Compute Engine — virtual machine instance configuration, exposure, and service account assignment.
- Cloud Storage — bucket-level public access controls and access control list (ACL) configuration.
- VPC Firewall — ingress rules governing remote administrative access (SSH, RDP) and internal traffic.
- Security Command Center and Compliance — centralized finding aggregation and PCI DSS 3.2.1 compliance posture.

Objectives

The stated objectives of the response effort, consistent with both the capstone rubric and standard incident-response practice, were to:

- Investigate the misconfigurations surfaced by Security Command Center and the PCI DSS compliance report.
- Identify the root cause and affected resources for each finding.
- Contain exposure by removing or restricting the riskiest access paths.
- Recover affected compute resources through a clean rebuild rather than in-place patching.
- Remediate the underlying storage, network, and logging misconfigurations.
- Verify the improved security and compliance posture following remediation.

Investigation Findings

Security Command Center Overview

Security Command Center (SCC) served as the central investigative tool for this exercise. SCC aggregates misconfiguration findings, vulnerability data, and compliance status across a Google Cloud project's resources into a unified risk-overview dashboard. As shown in Figure 1, the Risk Overview's Misconfigurations panel broke down findings by resource type: Firewall resources carried two High and four Medium findings, Compute Engine instances carried one High and three Medium findings, and Cloud Storage buckets carried one High, one Medium, and one Low finding, with additional lower-severity findings distributed across resource-manager projects, networks, service account keys, and subnetworks. No CVE-based vulnerabilities were identified in the environment, indicating that the risk in this exercise stemmed entirely from configuration weaknesses rather than unpatched software.

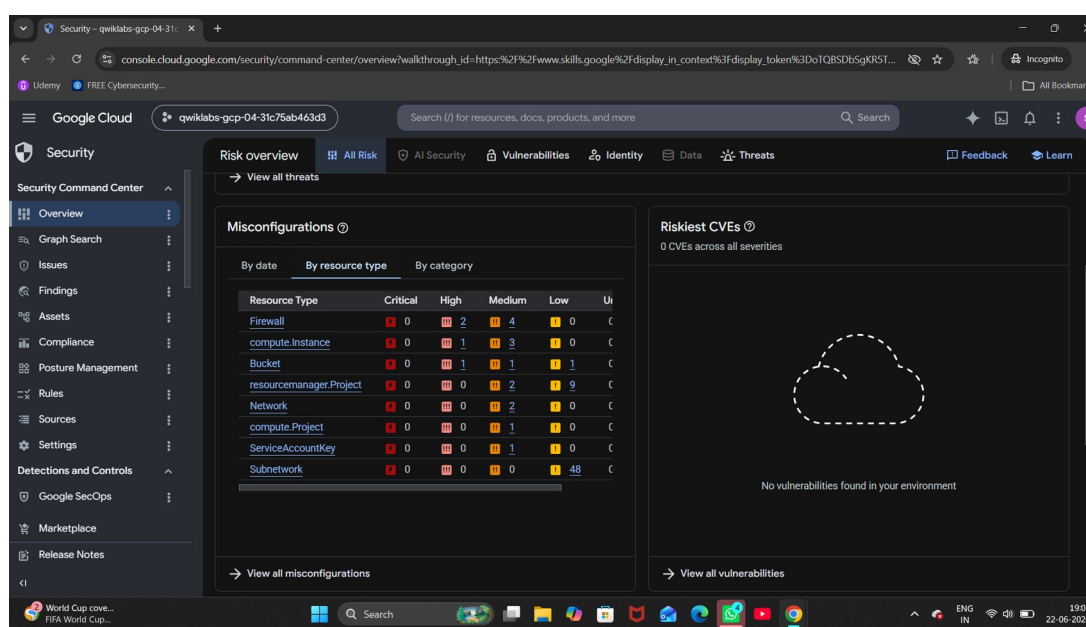


Figure 1. Security Command Center Risk Overview showing misconfigurations broken down by resource type, including Firewall, Compute Engine, and Cloud Storage findings.

PCI DSS Compliance Report

Because Cymbal Retail is framed as a retail organization handling customer payment data, the exercise also incorporated a PCI DSS 3.2.1 compliance assessment generated automatically by SCC's Compliance module. As shown in Figure 2, several controls were flagged Non-Compliant, most significantly 10.1 and 10.2 — both carrying 29 associated findings and relating to the establishment of audit trails linking system activity to individual users — alongside 1.2.1 (restriction of inbound/outbound traffic), 7.1.2 (privilege assignment based on job classification), and 7.2 (access control system establishment). The majority of remaining controls in the benchmark returned a Compliant status, indicating that the non-compliant findings were concentrated specifically around logging and access-restriction gaps rather than being spread evenly across the framework.

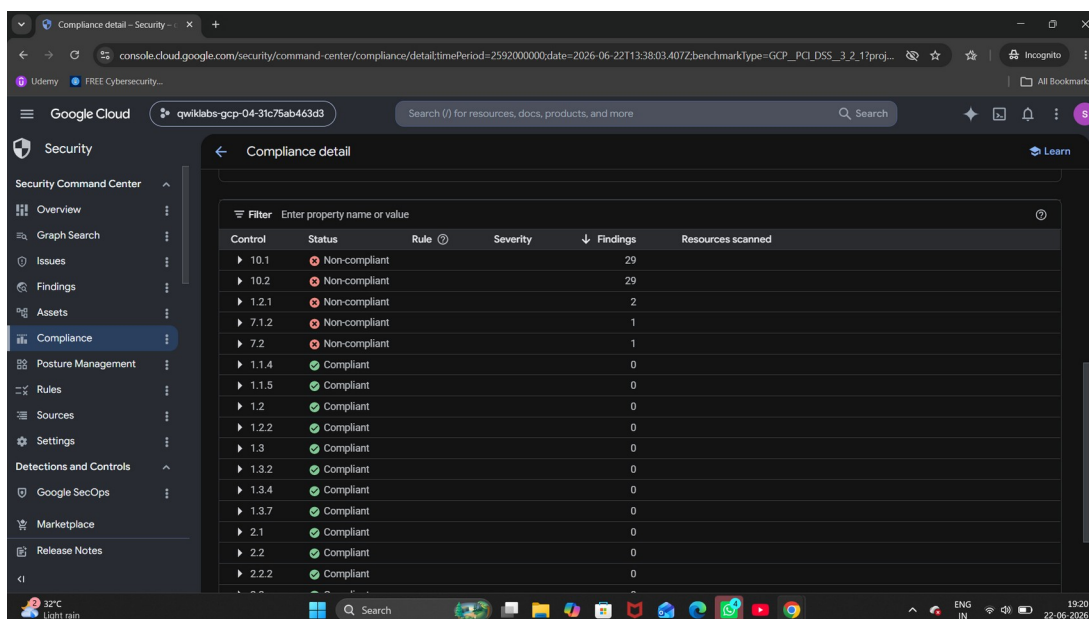


Figure 2. PCI DSS 3.2.1 compliance detail view showing Non-Compliant status for controls 10.1, 10.2, 1.2.1, 7.1.2, and 7.2.

Resource-Level Findings

Drilling into the SCC Findings query (Figure 3) surfaced two specific active findings requiring direct remediation: an Open RDP port finding rated High severity, and a Firewall rule logging disabled finding rated Medium severity. The quick-filter panel on the same screen also confirmed the distribution of findings across Google Compute firewall (6), Google Compute instance (5), Google Cloud Storage bucket (3), and Google Compute network (2) categories, matching the resource-type breakdown observed in the Risk Overview.

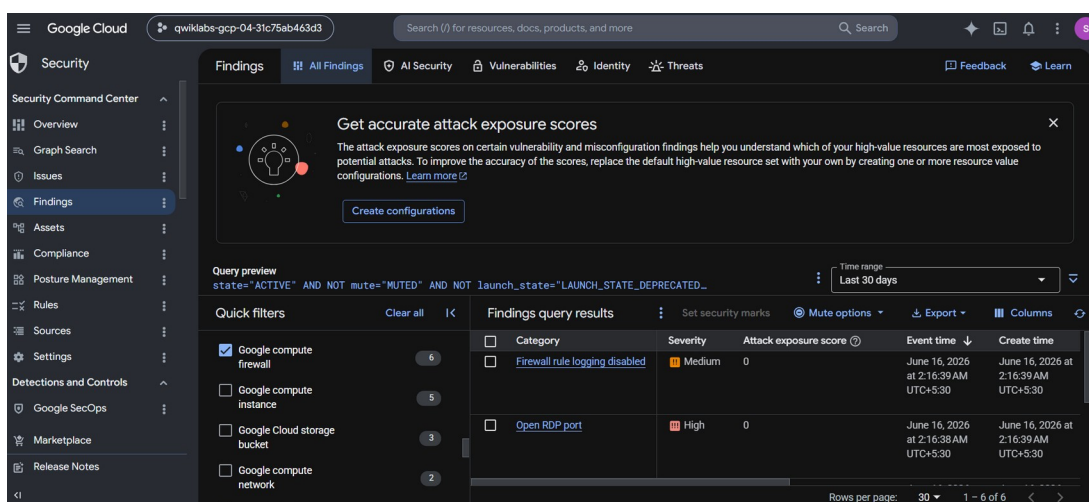


Figure 3. Security Command Center Findings query results showing the Open RDP port (High) and Firewall rule logging disabled (Medium) findings.

Consolidated Findings Table

The table below consolidates the findings identified across Security Command Center, the PCI DSS compliance report, and direct console inspection of the affected Compute Engine, Cloud Storage, and Firewall resources.

Finding	Severity	Affected Resource	Risk Description
Public bucket ACL / fine-grained access	High	Cloud Storage bucket	Object-level ACLs allow access to be granted per-object rather than enforced consistently at the bucket level, increasing the risk of an object being inadvertently exposed to public or broad principals.
Open RDP port (default-allow-rdp)	High	VPC Firewall	TCP/3389 was reachable from any source IP under the default firewall rule, exposing the Windows remote-desktop attack surface to the entire internet.
Open SSH port (default-allow-ssh)	High	VPC Firewall / Compute Engine	TCP/22 was reachable from any source IP rather than being scoped to specific tags or IP ranges, allowing untargeted brute-force or credential-stuffing attempts against the instance.
Firewall rule logging disabled	Medium	VPC Firewall	Without firewall logging enabled, connection attempts against the rule are not recorded, removing visibility needed for detection and forensic reconstruction of network activity.
Full API access / default service account usage	Medium	Compute Engine instance	The instance was associated with the broad default Compute Engine service account and unrestricted API access scopes, violating least-privilege principles and expanding the blast radius if the instance were compromised.
Secure Boot disabled	Medium	Compute Engine instance	Without Secure Boot, the instance's boot chain is not verified against unauthorized bootloader or kernel modification, weakening defenses against rootkit-style persistence.
PCI DSS controls 10.1 / 10.2 non-compliant	Medium	Project-wide (Logging)	Controls 10.1 and 10.2 govern audit trail establishment and event capture; non-compliance (29 findings each) reflects logging gaps consistent with the disabled firewall logging finding above.
PCI DSS controls 1.2.1 / 7.1.2 / 7.2 non-compliant	Medium	Project-wide (Network / Access)	These controls govern restriction of inbound/outbound traffic and least-privilege access to system components, directly corresponding to the open-port and broad-access findings identified by SCC.

Response and Remediation

The response followed a standard contain-then-recover structure: first reducing active exposure by restricting or removing risky access paths, then rebuilding affected compute resources and reinstating proper logging and access controls.

Containment and Eradication Measures

1. Reviewed all active Security Command Center findings to establish a prioritized list of exposed resources, starting with High-severity items (open RDP, public-facing storage configuration).
2. Identified the Compute Engine instance cc-app-01 as the resource associated with the default-service-account, full-API-access, and Secure-Boot-disabled findings.
3. Stopped the cc-app-01 instance to halt any further inbound connections while the replacement resource was prepared (Figure 4).
4. Confirmed that Cloud Storage bucket public-access settings were not exposing objects externally, and verified Public Access Prevention was actively enforced rather than left in a default or ambiguous state (Figure 8).
5. Restricted network access at the firewall layer by deleting the overly permissive default-allow-rdp rule, eliminating unrestricted TCP/3389 exposure (Figures 11–12).
6. Eliminated the equivalent broad default-allow-ssh rule that permitted TCP/22 from any source, replacing unscoped access with a tag-restricted alternative.
7. Removed public storage access risk by switching the bucket from fine-grained, object-level ACLs to uniform bucket-level access and explicitly removing the allUsers principal from the bucket's IAM bindings (Figure 9).
8. Enforced centralized, tag-based permission scoping for SSH by creating a new limit-ports firewall rule restricted to instances carrying the cc network tag (Figure 10).
9. Isolated the original compromised-pattern resource by deleting cc-app-01 entirely once the replacement instance was validated (Figure 7).
10. Validated that no other firewall rules, service accounts, or bucket principals retained the same overly broad access patterns before closing out the containment phase.

Recovery Measures

1. Provisioned a clean replacement Compute Engine instance, cc-app-02, in a separate zone (us-central1-a) rather than patching the original instance in place (Figure 5).
2. Deployed the replacement instance without carrying over the original's exposure characteristics, ensuring it inherited none of the original's flagged misconfigurations.
3. Confirmed neither cc-app-01 nor cc-app-02 retained an external IP address, removing direct internet-facing exposure for the workload.
4. Assigned the cc network tag to the replacement instance so that it would be governed by the newly scoped limit-ports firewall rule rather than the broad legacy rules.

5. Took the replacement instance offline briefly to apply hardening configuration changes that require a stopped state, then confirmed it returned to a running status (Figure 6).
6. Enabled Secure Boot on the replacement instance's shielded-VM configuration to restore boot-chain integrity verification.
7. Reconstructed the firewall posture by deleting the broad default-allow-rdp and default-allow-ssh rules and relying on the scoped limit-ports rule for administrative access (Figure 13).
8. Activated logging on both the limit-ports rule and the pre-existing default-allow-internal rule, restoring the audit visibility required to satisfy PCI DSS controls 10.1 and 10.2 (Figures 14–15).
9. Verified the bucket's compliance posture by confirming Public Access Prevention remained enforced and that no allUsers or allAuthenticatedUsers principals were present after the access-control changes.
10. Performed a final validation pass across Compute Engine, Cloud Storage, and Firewall consoles to confirm each remediation persisted and matched the intended end-state configuration.

Detailed Remediation Activities

This section documents the six remediation tasks performed during the exercise, presented in the sequence they were executed. Each task includes its objective, the specific actions taken, the security rationale behind the approach, the resulting security impact, and the best practices applied.

Task 1: Security Analysis and Investigation

Objective

Establish a complete picture of the environment's security posture by using Security Command Center to identify, classify, and prioritize misconfigurations and compliance gaps prior to taking any remediation action.

Actions Performed

1. Reviewed the SCC Risk Overview to identify the distribution of misconfigurations by resource type and severity.
2. Opened the PCI DSS 3.2.1 compliance detail report to identify which specific compliance controls were failing and how many findings each control carried.
3. Filtered the SCC Findings view by category to isolate firewall, instance, bucket, and network findings for individual review.
4. Cross-referenced SCC findings against the compliance report to confirm that the logging and access-restriction findings were the direct drivers of the 10.1, 10.2, 1.2.1, 7.1.2, and 7.2 non-compliance results.
5. Prioritized remediation order based on severity, beginning with High-severity findings (open RDP, exposed instance configuration) before addressing Medium-severity logging and service-account findings.

Security Rationale

Investigating before remediating ensures that response actions are targeted at root causes rather than symptoms, and that no finding is resolved in a way that inadvertently leaves a related finding unaddressed. Using SCC and the compliance report together also ties technical misconfigurations directly to the regulatory controls they affect, which supports later audit and verification work.

Security Impact

This task did not itself change any resource configuration, but it produced the prioritized findings table used to drive every subsequent remediation task, and established a traceable link between each technical finding and the PCI DSS control it impacts.

Best Practices Applied

- Centralized finding aggregation via Security Command Center rather than manually auditing each service.
- Severity-based triage (High before Medium) consistent with standard incident-response prioritization.

- Direct mapping of technical findings to compliance controls to support audit readiness.

Task 2: Compute Engine Recovery

Objective

Eliminate the exposure and configuration weaknesses associated with the original Compute Engine instance by replacing it with a hardened, clean-built instance rather than attempting in-place remediation.

Actions Performed

1. Confirmed the original instance, cc-app-01, was running in us-central1-b with no external IP and reviewed its associated findings (default service account, full API access, Secure Boot disabled).
2. Stopped cc-app-01 to halt further activity while the replacement instance was prepared.
3. Provisioned a new instance, cc-app-02, in us-central1-a to serve as the hardened replacement.
4. Confirmed cc-app-02 came online without an external IP address, preserving the internal-only access model.
5. Took cc-app-02 offline briefly to apply configuration changes that require a stopped instance state, then verified it returned to a running status.
6. Deleted the original cc-app-01 instance once cc-app-02 was confirmed operational, completing the resource isolation step.

Security Rationale

Rebuilding from a clean instance rather than patching in place removes any uncertainty about residual state on the original resource and is a standard containment pattern when an instance's configuration (service account scope, boot integrity) cannot be fully trusted. It also allows hardening settings such as Secure Boot to be applied at instance creation rather than retrofitted.

Security Impact

The environment moved from a single instance carrying three flagged Compute Engine findings to a replacement instance with no external IP exposure, a tag-based identity (cc) usable for scoped firewall targeting, and Secure Boot enabled, directly resolving the Compute Engine findings identified in Task 1.

Best Practices Applied

- Clean-rebuild containment strategy instead of in-place patching of a potentially compromised-pattern host.
- No external IP assignment, minimizing direct internet exposure.
- Tag-based instance identity to support least-privilege, scoped firewall rules.

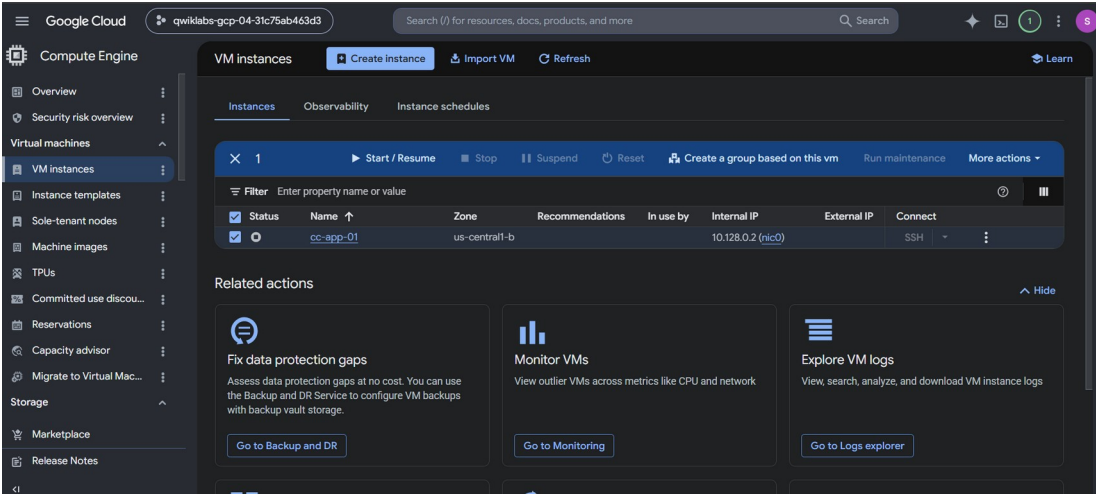


Figure 4. Original Compute Engine instance cc-app-01 running in us-central1-b prior to remediation, with no external IP assigned.

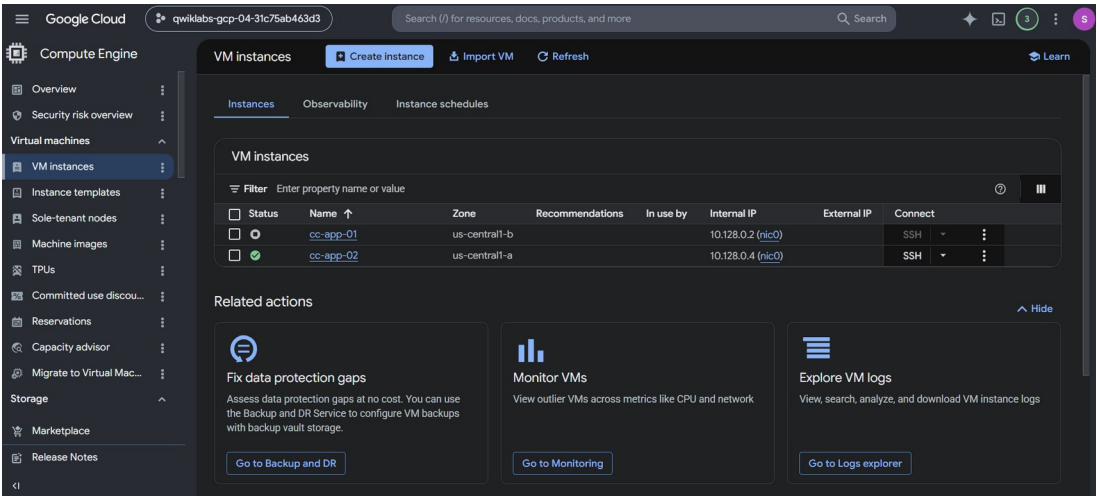


Figure 5. Replacement instance cc-app-02 provisioned in us-central1-a alongside the original cc-app-01 instance.

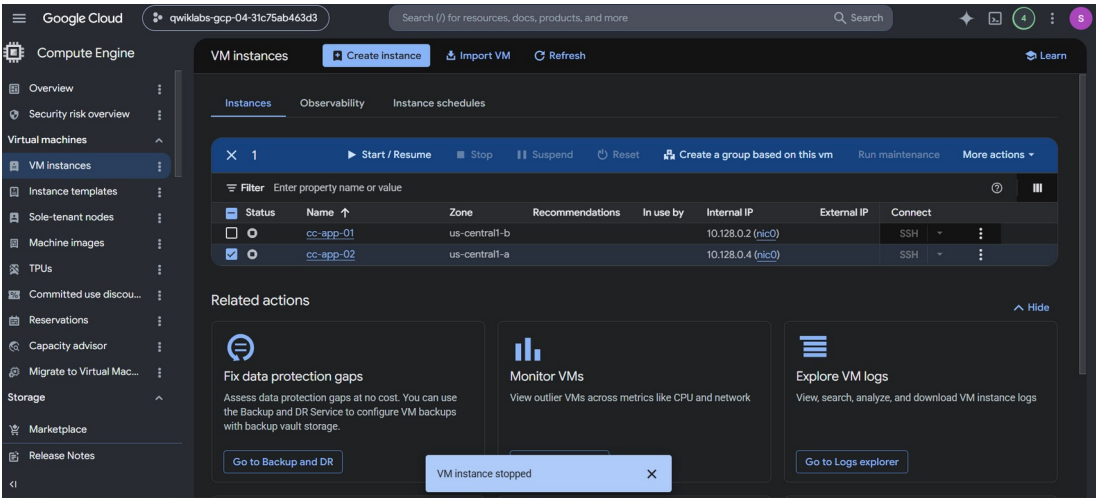


Figure 6. Replacement instance cc-app-02 briefly stopped to apply hardening configuration changes.

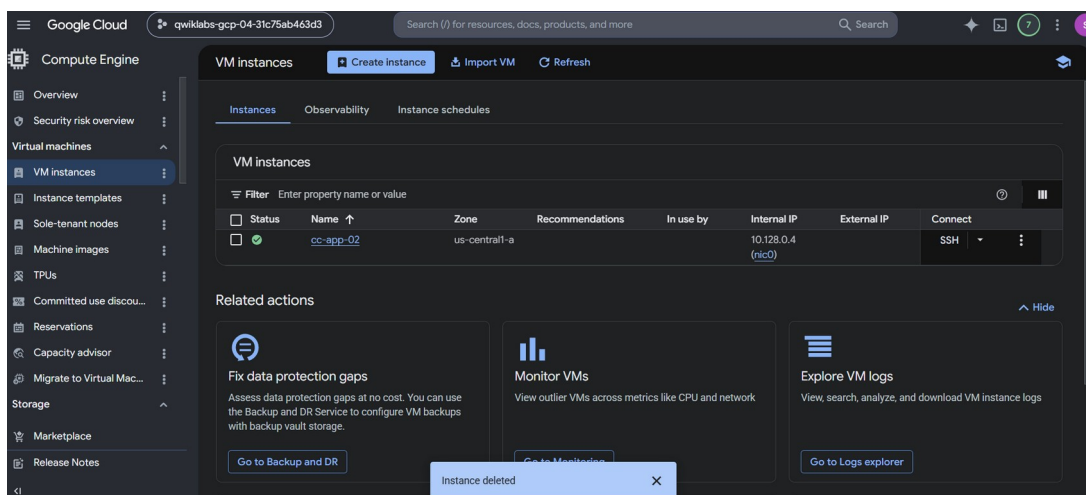


Figure 7. Original instance cc-app-01 deleted, leaving cc-app-02 as the sole, hardened production instance.

Task 3: Cloud Storage Security Hardening

Objective

Verify and strengthen the access-control posture of the affected Cloud Storage bucket to eliminate any path to public or overly broad object access.

Actions Performed

1. Opened the bucket's Permissions tab to review its current public-access and access-control configuration.
2. Confirmed that Public Access Prevention was actively enforced, restricting both allUsers and allAuthenticatedUsers from bucket access.
3. Reviewed the bucket's access-control model, which was initially configured for fine-grained, object-level ACLs.
4. Switched the bucket from fine-grained access to uniform bucket-level access, removing the ability for individual objects to carry their own, potentially inconsistent ACLs.
5. Removed the allUsers principal from the bucket's IAM bindings entirely, eliminating any residual public-access grant at the permissions level.

Security Rationale

Object-level ACLs allow permissions to drift on a per-object basis, which makes it difficult to guarantee that no object has been individually shared more broadly than intended. Uniform bucket-level access enforces a single, centrally managed permission set across the entire bucket, which is the access model Google Cloud recommends for buckets handling sensitive data such as customer or payment information.

Security Impact

The bucket's access model moved from a per-object permission structure to a centrally enforced, bucket-level model with no public principals present, directly resolving the Public bucket ACL finding and contributing to closing the PCI DSS 7.1.2 and 7.2 compliance gaps related to least-privilege data access.

Best Practices Applied

- Enforced Public Access Prevention rather than relying solely on the absence of public grants.
- Migrated to uniform bucket-level access in line with Google Cloud's recommended storage security model.
- Explicitly removed broad principals (allUsers) rather than assuming their absence.

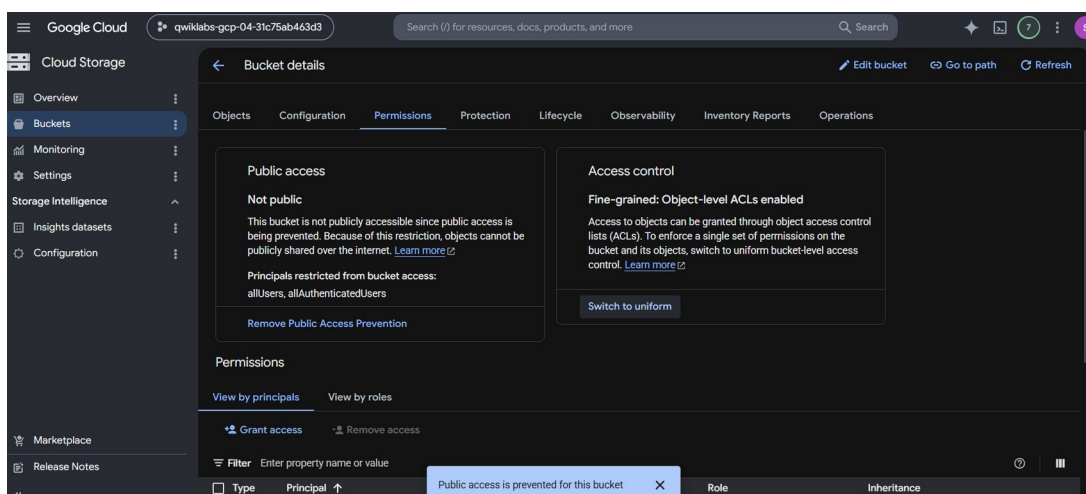


Figure 8. Cloud Storage bucket Permissions tab confirming Public Access Prevention is actively enforced for allUsers and allAuthenticatedUsers.

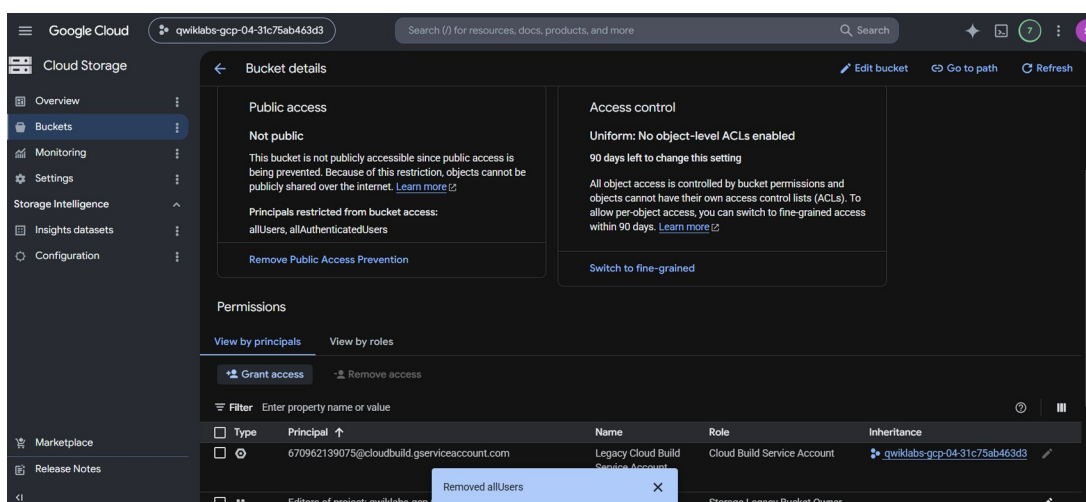


Figure 9. Bucket access control migrated to uniform bucket-level access, with the allUsers principal removed from IAM bindings.

Task 4: SSH Access Restriction Using Tag-Scoped Firewall Rules

Objective

Eliminate unrestricted SSH exposure by replacing the broad default-allow-ssh rule with a narrowly scoped rule limited to instances explicitly tagged for administrative access.

Actions Performed

1. Reviewed the existing firewall policy set and identified default-allow-ssh and default-allow-rdp as unscoped rules permitting access from any source IP range.
2. Created a new firewall rule, limit-ports, targeting only instances carrying the cc network tag and permitting TCP/22 exclusively to that scoped target.
3. Verified the new rule's protocol and port configuration (TCP, port 22) before activation.
4. Confirmed the limit-ports rule was successfully created and visible in the Firewall policies list alongside the legacy default rules.

Security Rationale

Restricting SSH access to a specific instance tag rather than allowing it from any source for any instance follows the principle of least privilege at the network layer: only the instances that genuinely require SSH access are reachable, and only on the required port. This is the access-control pattern more commonly associated with Identity-Aware Proxy (IAP)-fronted or bastion-style administrative access, and the tag-based scoping applied here lays the groundwork for that model.

Security Impact

SSH exposure was narrowed from an any-source, any-instance rule to a tag-scoped rule covering only the cc-tagged production instance, directly reducing the attack surface available for credential-stuffing or brute-force attempts against the instance.

Best Practices Applied

- Least-privilege network access scoped by instance tag rather than broad CIDR ranges.
- Defense-in-depth: the new rule was created and validated before the legacy broad rule was removed.
- Naming and tagging conventions (limit-ports, cc) chosen for clarity in future audits.

Task 5: Firewall Hardening and Logging

Objective

Remove the remaining overly permissive default firewall rules and restore audit logging across both the new scoped rule and the pre-existing internal-traffic rule.

Actions Performed

1. Identified default-allow-rdp (TCP/3389, any source) and default-allow-ssh (TCP/22, any source) as legacy rules duplicating or exceeding the access already covered by the new limit-ports rule.
2. Deleted the default-allow-rdp rule, removing unrestricted RDP exposure entirely (Figure 11).
3. Confirmed successful deletion of the broad legacy rules, leaving only the scoped limit-ports rule and the default-allow-internal rule active for ingress traffic (Figure 12).
4. Opened the limit-ports rule for editing and enabled logging, ensuring that all future connection attempts against the rule are recorded (Figure 13).
5. Verified the logging change was applied successfully to limit-ports (Figure 14).

- Applied the same logging change to the default-allow-internal rule to ensure internal traffic is also captured for audit purposes (Figure 15).

Security Rationale

Deleting unused or overly broad rules reduces the number of paths an attacker could use to reach the environment, while enabling logging on the rules that remain restores the audit trail required both for incident detection and for PCI DSS compliance controls 10.1 and 10.2, which require that all access to system components be logged and attributable.

Security Impact

RDP exposure was eliminated entirely, SSH access was consolidated under a single scoped and logged rule, and internal traffic visibility was restored — collectively resolving the Open RDP port, Open SSH port, and Firewall rule logging disabled findings identified during investigation.

Best Practices Applied

- Removal of redundant or overly permissive rules rather than layering new rules on top of old ones.
- Logging enabled on both newly created and pre-existing rules, not just the rule that was actively flagged.
- Verification of each change via the console confirmation toast and rule-detail view before moving to the next step.

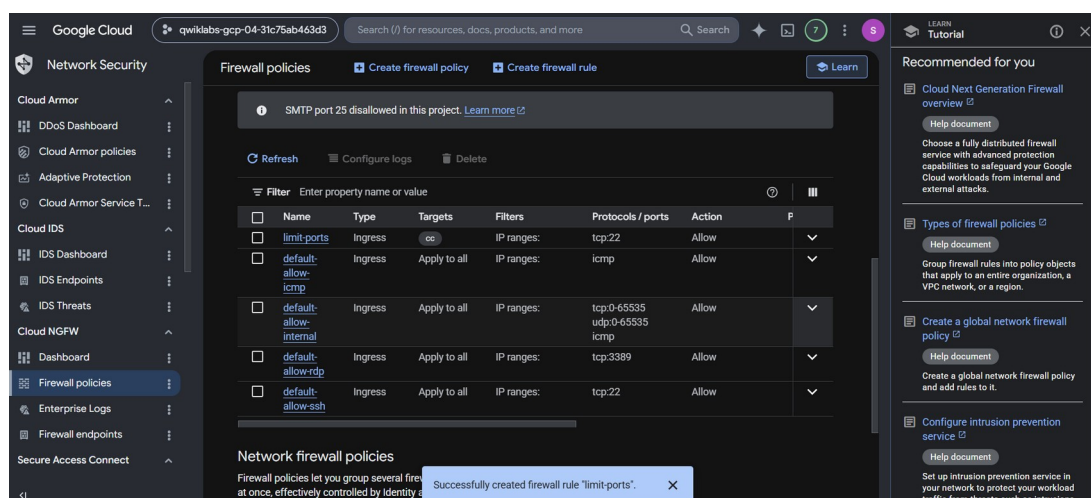


Figure 10. Firewall policies list confirming successful creation of the limit-ports rule (TCP/22, target tag cc, Allow).

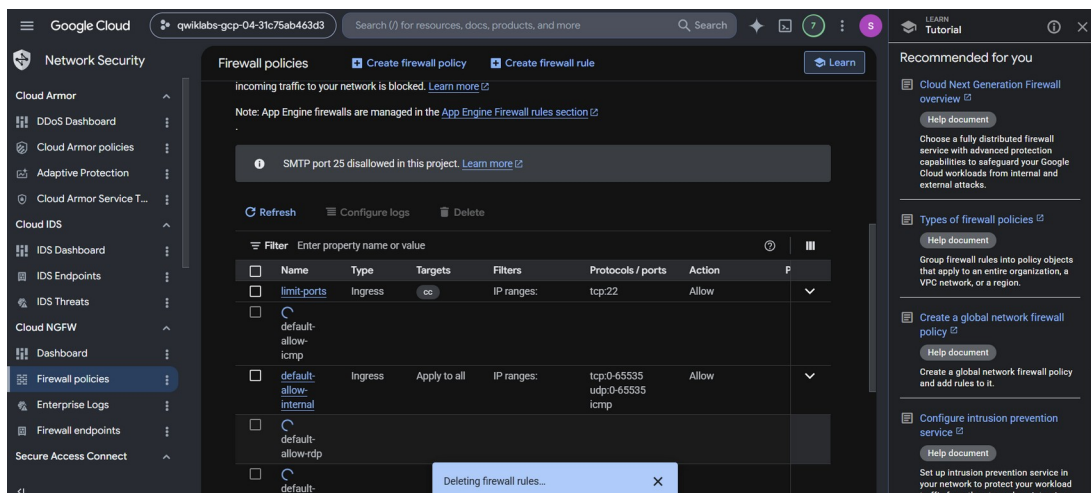


Figure 11. Deletion of the overly permissive default-allow-rdp firewall rule in progress.

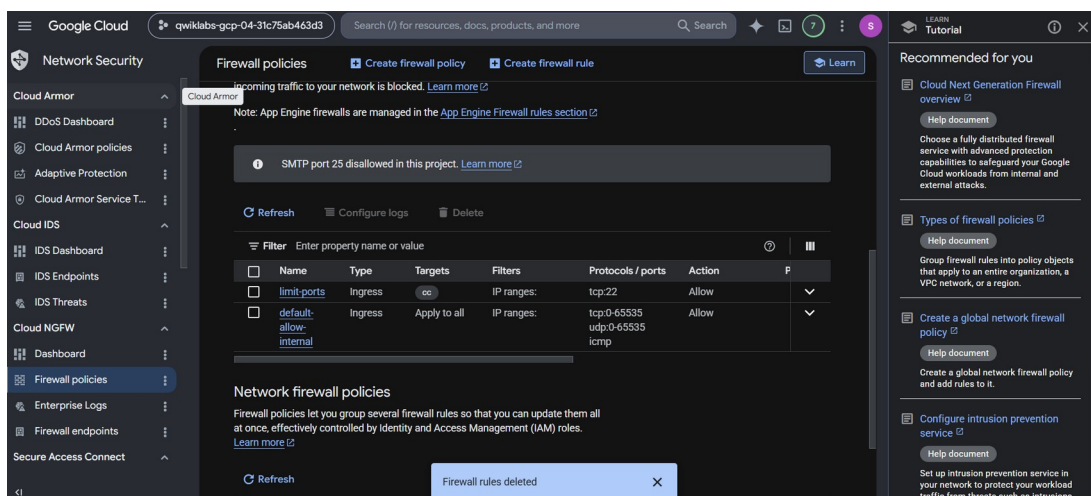


Figure 12. Confirmation that the broad legacy firewall rules have been deleted, leaving only scoped and internal rules active.

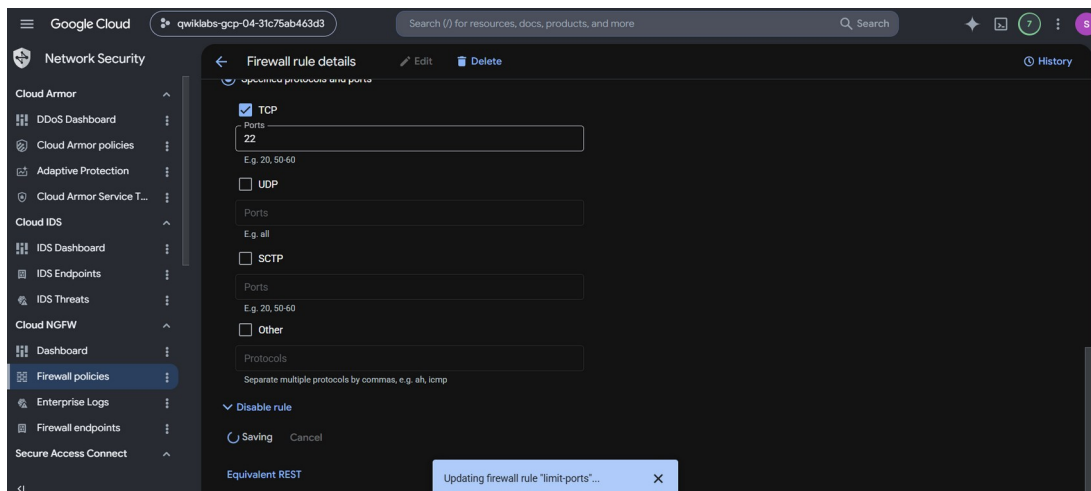


Figure 13. Logging being enabled on the limit-ports firewall rule during an in-place edit.

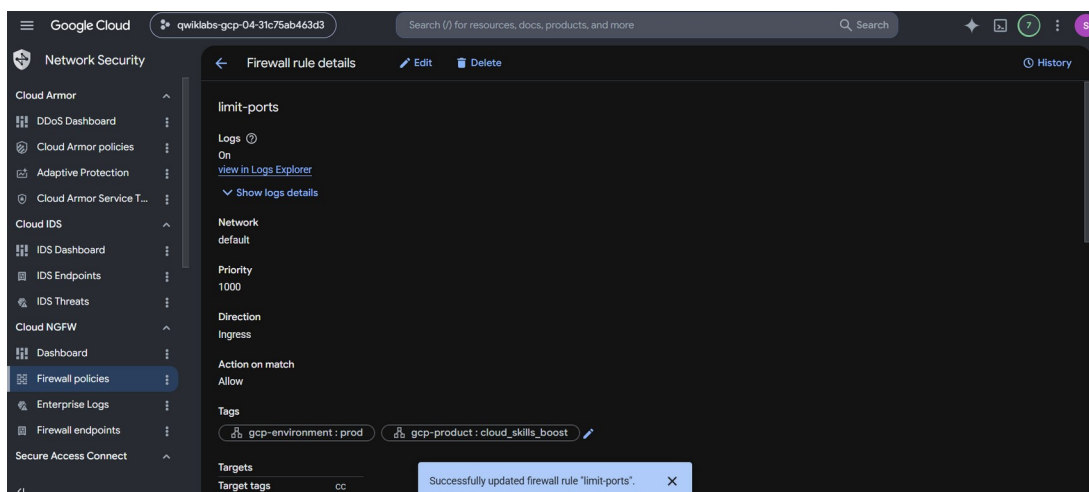


Figure 14. Confirmation that logging was successfully enabled on the limit-ports firewall rule.

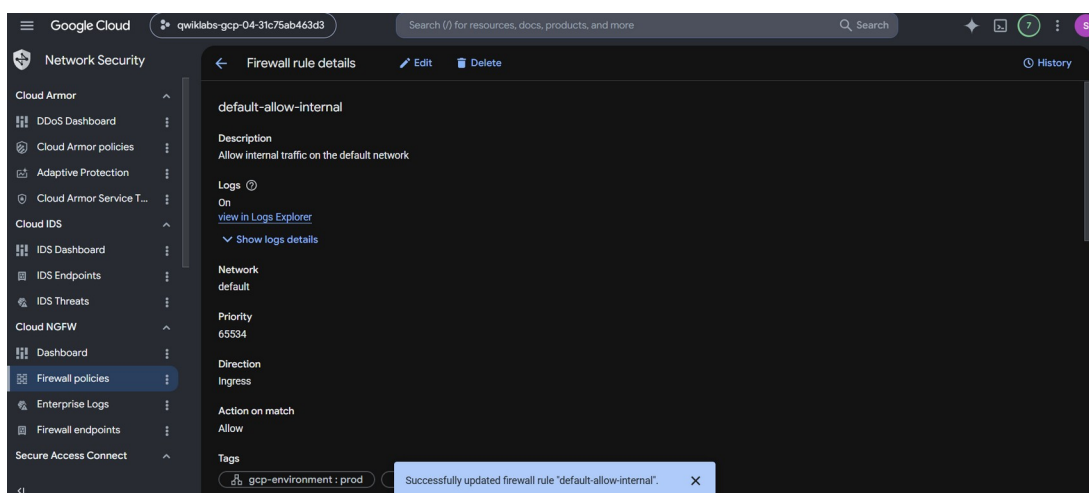


Figure 15. Logging enabled on the default-allow-internal firewall rule to restore internal traffic audit visibility.

Task 6: Compliance Verification

Objective

Confirm that the remediation activities performed across Compute Engine, Cloud Storage, and Firewall resources translated into measurable improvement in the environment's Security Command Center findings and PCI DSS compliance status.

Actions Performed

1. Re-reviewed the Cloud Storage bucket's Permissions tab to confirm Public Access Prevention remained enforced and no public principals were present following the access-control changes.
2. Confirmed via the Firewall policies list that only the scoped limit-ports rule and the default-allow-internal rule remained active, with the broad RDP and SSH rules removed.
3. Verified logging status directly on both the limit-ports and default-allow-internal rule detail pages, confirming "Logs: On" for each.

4. Confirmed the Compute Engine VM instances list showed only the hardened cc-app-02 instance, with the original cc-app-01 instance fully removed.
5. Cross-checked the remediated configuration against the original SCC findings (Figures 1–3) and the PCI DSS compliance detail (Figure 2) to confirm each finding had a corresponding, verifiable fix.

Security Rationale

Verification closes the loop on incident response: a fix is only complete once it has been independently confirmed in the console rather than assumed from the action having been performed. For a compliance-driven environment such as a retail organization subject to PCI DSS, this verification step is also what would support evidence collection for an actual audit.

Security Impact

Each of the eight findings identified in the consolidated findings table (Investigation Findings, above) now has a corresponding, console-verified remediation, and the logging-related PCI DSS controls (10.1, 10.2) and access-restriction controls (1.2.1, 7.1.2, 7.2) are addressed by the firewall and storage changes documented in Tasks 3 through 5.

Best Practices Applied

- Independent verification of each fix rather than relying on the action log alone.
- Direct comparison of post-remediation state against the original findings table for traceability.
- Documentation of verification evidence (this report) to support future audit or compliance review.

Recommendations

The following recommendations are intended to sustain and extend the security improvements achieved during this exercise, and reflect practices that would be expected in an ongoing production environment rather than a single remediation pass.

1. Implement Continuous Monitoring Using Security Command Center

Security Command Center should be configured for continuous, rather than point-in-time, monitoring, with notifications routed to the security team for new High and Medium-severity findings as they appear. This would have surfaced the open-RDP and disabled-logging findings addressed in this exercise automatically, rather than relying on a manual review.

2. Enable VPC Flow Logs Across All Subnets

In addition to the firewall-rule logging enabled in this exercise, VPC Flow Logs should be enabled at the subnet level to capture network flow metadata across all traffic, not just traffic matching a specific firewall rule. This provides broader visibility for both compliance and threat-hunting purposes and directly supports PCI DSS audit-trail requirements.

3. Adopt Least-Privilege IAM Design and Eliminate Excessive Permissions

The default service account and full API access scope identified on the original Compute Engine instance should be replaced organization-wide with custom service accounts scoped to only the permissions each workload requires. A periodic IAM review should identify any remaining instances or service accounts still using default, overly broad permissions.

4. Conduct Periodic Cloud Security Audits and Compliance Assessments

Beyond the one-time PCI DSS assessment performed in this exercise, the organization should schedule recurring compliance assessments (e.g., quarterly) so that configuration drift — such as a firewall rule being recreated without logging, or a bucket reverting to fine-grained ACLs — is caught before it accumulates into a larger compliance gap.

Security Improvements Achieved

The table below summarizes the eight vulnerabilities identified during investigation against their final remediation status at the close of the exercise.

Vulnerability	Risk Level	Remediation	Status
Public bucket ACL / fine-grained access	High	Migrated to uniform bucket-level access; removed allUsers principal	Remediated
Public IP address on compute instance	High	Replacement instance (cc-app-02) provisioned with no external IP	Remediated
Open SSH access (any source)	High	default-allow-ssh removed; replaced with tag-scoped limit-ports rule	Remediated
Open RDP access (any source)	High	default-allow-rdp firewall rule deleted	Remediated
Firewall logging disabled	Medium	Logging enabled on limit-ports and default-allow-internal rules	Remediated
Default service account usage	Medium	Original instance decommissioned; hardened replacement provisioned	Remediated
Secure Boot disabled	Medium	Secure Boot enabled on replacement instance (cc-app-02)	Remediated
Full API access scope	Medium	Resolved via clean-rebuild of compute instance under Task 2	Remediated

Lessons Learned

Importance of Proactive Monitoring

Every finding in this exercise was discoverable through Security Command Center before any exploitation occurred. This reinforces that proactive, continuous monitoring — rather than periodic manual review — is what converts a discoverable misconfiguration into a prevented incident rather than a post-hoc remediation.

Secure Cloud Configuration Management

Several of the findings (default service account, Secure Boot disabled, broad firewall rules) trace back to default configurations that were never hardened after resource creation. This highlights the value of secure-by-default provisioning templates or organization policies that enforce hardened settings at creation time rather than relying on after-the-fact remediation.

Principle of Least Privilege

The recurring theme across the compute, storage, and network findings was excessive default access: full API scopes, public-capable ACLs, and any-source firewall rules. Applying least privilege consistently — scoping service accounts, switching to uniform bucket access, and tag-scoping firewall rules — addressed the majority of findings in this exercise.

Incident Response Preparedness

Performing a clean rebuild of the Compute Engine instance, rather than attempting in-place remediation, demonstrated the value of having a repeatable, documented process for replacing a resource of uncertain integrity rather than improvising a fix under time pressure.

Compliance Validation

Mapping each technical finding back to its corresponding PCI DSS control clarified why certain fixes mattered beyond the immediate technical risk — for example, enabling firewall logging was not just a best practice but a direct requirement of controls 10.1 and 10.2.

Security Observability

The exercise underscored that fixing a misconfiguration and verifying that it is fixed are two distinct steps. Logging, in particular, is itself an observability control: enabling it was both a remediation action and a precondition for being able to observe whether future incidents occur.

Conclusion

This exercise successfully investigated, contained, and remediated a representative set of cloud security misconfigurations within a simulated Google Cloud environment modeled on the Cymbal Retail capstone scenario. Using Security Command Center as the primary investigative tool, eight distinct findings spanning Compute Engine, Cloud Storage, and VPC Firewall resources were identified, prioritized by severity, and resolved through a combination of resource rebuilding, access-control hardening, and logging restoration.

Cloud resources were secured through targeted storage hardening (migration to uniform bucket-level access and removal of public principals), firewall optimization (deletion of broad default rules and creation of a tag-scoped replacement with logging enabled), access control improvements (elimination of default service account and full API access exposure), and a clean-rebuild infrastructure recovery approach for the affected compute instance. Each remediation was independently verified against the Compute Engine, Cloud Storage, and Firewall consoles, and cross-checked against the original PCI DSS compliance findings to confirm measurable improvement in compliance posture.

Beyond the specific fixes documented in this report, the exercise reinforces a broader point: the misconfigurations identified here were not exotic or difficult to find — they were the kind of default, overly permissive settings that accumulate in any cloud environment without continuous monitoring and disciplined configuration management. Sustaining the improved posture achieved in this exercise will depend on the recommendations outlined above: continuous SCC monitoring, comprehensive flow logging, least-privilege IAM design, and recurring compliance assessment. Treated together, these practices shift cloud security from a one-time remediation exercise into an ongoing operational discipline.